

SMART RETROFIT METERING CIE

General Design • Detail Design • Document Technique • Code requis • Algorithmes • Dashboards

Solution compteur à carte connectée sans changement fort de l'existant

Champ	Valeur
Client cible	CIE - Compagnie Ivoirienne d'Electricite
Version	1.0 - dossier de cadrage DSI / fournisseur
Date	17/08/2026
Hypothese majeure	Retrofit sans remplacement massif des compteurs existants ; compatibilite a qualifier par modele.
Objectif	Suivi temps reel, alerte credit faible, tele-recharge, supervision CIE, support distant et incidents.

Ce dossier propose une trajectoire industrialisable par la DSI : le compteur à carte existant reste en place, un module retrofit qualifié est ajouté lorsque techniquement possible, les données et les API restent sous la gouvernance de la CIE, et le support est piloté à distance via un modèle L1/L2/L3.

1. Synthèse executive

La solution cible transforme le compteur à carte en compteur communicant sans refonte lourde du parc installé. Elle repose sur un dongle ou un module retrofit, une connectivité mobile sécurisée, une plateforme IoT/HES/MDMS, une orchestration de paiement, des notifications client et un cockpit CIE pour le pilotage de la consommation, des incidents et des alertes.

- Client : consulter solde, consommation, autonomie prévisionnelle, alertes avant coupure, statut de recharge et historique.
- CIE : suivre le parc en temps quasi réel, détecter les compteurs hors ligne, analyser les incidents, piloter les recharges, prioriser les interventions.
- DSI : conserver la maîtrise des API, de la sécurité, des données, du monitoring, de l'exploitation et de la trajectoire produit.
- Fournisseurs : fournir HES/MDMS, dongle, firmware ou connecteurs, sous les exigences de réversibilité et d'audit.

2. General Design

2.1 Principes directeurs

ID	Principe	Description
P1	Ne pas remplacer massivement les compteurs	Le compteur existant reste la source métier, le retrofit ajoute la connectivité et l'interface de lecture/commande.
P2	Qualifier avant généraliser	Chaque modèle compteur doit être qualifié : port optique, RS485, CIU, clavier, DLMS/COSEM, protocole constructeur, STS, contraintes de sécurité.
P3	Préserver le SI de prépaiement	La génération du token ou de l'ordre de recharge reste dans le système existant ou dans un service d'orchestration connecté.
P4	Rendre le token invisible quand possible	Le client achète du crédit ; la plateforme transmet le token ou la commande au compteur si la compatibilité est confirmée.
P5	Fallback obligatoire	Si injection automatique impossible, le token reste visible dans l'application/SMS/USSD afin de ne pas bloquer le service.
P6	Pilotage à distance	Tout incident doit remonter avec identifiant compteur, device, zone, horodatage, type d'incident, statut et correlation_id.
P7	Sécurité by design	Identité device, certificats, chiffrement, signature de commandes, HSM/PKI, supervision SOC et journalisation inaltérable.

2.2 Architecture cible E2E

Couche	Composant	Rôle
Terrain	Compteur existant + dongle retrofit	Lire crédit/index/événements ; injecter commande si compatible ; store-and-forward local.
Connectivité	SIM/eSIM M2M, APN privé, TLS	Acheminer messages MQTT/HTTPS de manière sécurisée ; gérer zones faible couverture.
IoT Platform	MQTT broker, device registry, command queue	Authentifier devices, recevoir telemetry, envoyer commandes, gérer ACK et retries.
HES	Head-End System spécialisée	Dialoguer avec compteur ou dongle, normaliser protocoles, piloter commandes et événements.
MDMS	Meter Data Management	Historiser, valider, estimer, agréger données de consommation et événements.
Paiement	Mobile Money / Vending / STS	Recevoir paiement, générer token/ordre, tracer statut de bout en bout.
Expérience	App client, USSD/SMS, portail support	Suivi conso, alerte, recharge, statut incident, selfcare.
Exploitation	NOC/SOC/ITSM	Supervision, incidents, tickets, runbooks, SLA, cyber alertes.

2.3 Flux cible de télé-recharge

Le flux cible doit fonctionner en deux modes : mode automatique lorsque le compteur accepte une injection distante, et mode assisté lorsque le compteur impose une saisie clavier par le client.

Etape	Action	Système	Preuve / contrôle
1	Client initie paiement	Application CIE, USSD, agence, Mobile Money	payment_id créé ; montant ; contrat ; meter_id.
2	Paiement confirmé	Mobile Money / agrégateur	Signature/verif ; statut CONFIRMED ; anti double paiement.
3	Génération recharge	Vending/STs ou service prépaiement	token_id ou command_id ; valeur énergie ; expiration.
4	Orchestration commande	Command Service	correlation_id ; idempotency_key ; choix mode automatique/fallback.
5	Transmission terrain	IoT/HES vers dongle/compteur	commande signée ; séquence ; timeout ; retry.
6	ACK appareil	Dongle/compteur	ACK technique ; ACK métier si crédit appliqué.
7	Confirmation client	Notification push/SMS/WhatsApp/USSD	Success, pending ou fallback avec token visible.
8	Audit complet	MDMS/DB/Audit log	journal immuable ; statut final ; preuves support.

3. Detail Design

3.1 Modules fonctionnels

Module	Responsabilite	Donnees principales
Device Registry	Identite device, association meter_id, certificat, firmware, statut	device_id, meter_id, cert_id, firmware_version, status, last_seen
Telemetry Ingestion	Reception index, credit, tension, courant, evenements	telemetry_id, timestamp, kwh_total, credit_estimated, event_type
Credit Forecast	Calcul d'autonomie previsionnelle et seuils d'alerte	daily_avg, remaining_credit, autonomy_days, risk_level
Recharge Orchestrator	Paieement, token, commande, ACK, notification	payment_id, token_id, command_id, correlation_id, status
Incident Engine	Classification incident et creation ticket	incident_id, severity, category, zone, SLA_clock
Notification Center	SMS, Push, WhatsApp, email, USSD	customer_id, channel, consent, status, retry_count
Support Console	Vue 360 client/compteur/recharge/incident	timeline, tickets, work_orders, final_status
CIE Cockpit	KPI DSI, metier, reseau, fraude, SLA	availability, offline_rate, recharge_success, tamper_rate

3.2 Regles de gestion client

ID	Regle	Detail
RG-C-001	Alerte credit faible	Declencher si autonomie <= seuil client ou seuil CIE par default. Message non anxiogene et actionnable.
RG-C-002	Alerte credit critique	Declencher si autonomie estimee <= 24h, avec bouton recharge rapide ou code USSD.
RG-C-003	Anti-spam	Ne pas envoyer plus d'une alerte de meme type par fenetre configuree, sauf passage a un niveau de criticite superieur.
RG-C-004	Autorecharge	Disponible uniquement apres consentement explicite, plafond montant/jour/mois, canal Mobile Money valide.
RG-C-005	Fallback token	Si injection automatique echoue apres N tentatives, afficher/envoyer token au client avec consignes simples.
RG-C-006	Transparence statut	Afficher : paiement recu, recharge en cours, compteur credite, token a saisir, echec rembourser/a traiter.

3.3 Regles de gestion CIE / support

ID	Regle	Detail
RG-S-001	Compteur offline	Si aucun heartbeat depuis duree configuree, classer offline ; verifier zone et operateur ; creer ticket si depassement seuil.
RG-S-002	Recharge payee non creditee	Tracer la chaine payment_id -> recharge_id -> command_id -> HES -> device -> ACK ; qualifier cause.
RG-S-003	Fraude potentielle	Combiner tamper event, baisse brutale consommation, ouverture boitier, incoherence tension/courant.
RG-S-004	Incident massif	Si cluster geographique ou operateur impacte, ouvrir incident majeur et basculer communication client.
RG-S-005	Priorisation terrain	Prioriser clients vulnerables, sites critiques, incidents securite, zones a forte recurrence, SLA le plus proche.
RG-S-006	Cloture prouvee	Cloture seulement avec statut technique, action realisee, horodatage, acteur et preuve telemetry ou photo terrain.

4. Document technique

4.1 Stack cible

Domaine	Technologie / code requis	Justification
Firmware dongle	C/C++, FreeRTOS, drivers UART/RS485/optique, modem, TLS, OTA	Temps reel, faible memoire, robustesse terrain.
IoT messaging	MQTT sur TLS, topics par tenant/zone/device	Leger, adapte devices, QoS, store-and-forward.
Backend critique	Java Spring Boot	Transactions, audit, idempotence, securite, integration SI.
API/BFF/Digital	TypeScript Node.js ou Java selon standard DSI	Exposition app client, support, portail, integration rapide.
Frontend	React TypeScript	Dashboards client, support, NOC, SOC.
Data	PostgreSQL, TimescaleDB, Kafka, Data Lake, Python/Spark	Transactionnel + serie temporelle + analytique.
SecOps	IAM, PKI, HSM, Vault, SIEM/SOC	Certificats, secrets, crypto, surveillance.
DevSecOps	GitLab CI, SAST/SCA, container scan, Kubernetes	Industrialisation, qualite, securite, deploiement repetable.

4.2 Modele de donnees minimal

Table	Champs cles	Usage
customer	customer_id, phone_hash, contract_id, consent_flags	Identite client et consentements de notifications/autorecharge.
meter	meter_id, model, protocol, firmware, location_id, status	Referentiel compteur existant.
device	device_id, meter_id, sim_id, cert_id, firmware_version, status	Dongle retrofit.
telemetry	telemetry_id, meter_id, timestamp, kwh_total, credit_balance, voltage, event_type	Flux temps reel et historique.
payment	payment_id, provider, amount, status, confirmed_at	Flux financier.
recharge	recharge_id, payment_id, token_hash, energy_value, status	Recharge/token et statut metier.
command	command_id, device_id, type, payload_hash, status, retry_count, ack_at	Commande terrain.

incident	incident_id, meter_id, type, severity, status, sla_due_at	Ticket exploitation.
audit_log	correlation_id, actor, action, before_hash, after_hash, timestamp	Traçabilité et audit.

5. Algorithmes

ALG-01 Calcul autonomie credit

- 1. Entrer remaining_credit_fcfa, historique de consommation journalière, prix/kWh applicable, seuils CIE et seuils client.
- 2. Nettoyer historique : supprimer jours incomplets, valeurs nulles, doublons telemetry, jours incident reseau majeur.
- 3. Calculer la consommation moyenne glissante sur 7 jours, puis sur 14 jours si les données sont insuffisantes.
- 4. Convertir le crédit restant en énergie restante si un tarif applicable est disponible ; sinon, utiliser la dépense moyenne en FCFA/jour.
- 5. Calculer autonomie = credit_rest / moyenne_journaliere ; borner à 0 si crédit négatif ou nul.
- 6. Classer : NORMAL > 7 jours, WATCH <= 7, LOW <= 3, CRITICAL <= 1, CUT_RISK <= 0.125.
- 7. Appliquer l'anti-spam : ne pas renvoyer au même niveau si déjà notifié récemment sauf aggravation.
- 8. Emettre evenement LOW_CREDIT avec risk_level, autonomy_days, confidence_score, recommended_amount.

ALG-02 Recharge automatique avec idempotence

- 1. Recevoir le callback de paiement signé ; vérifier la signature du fournisseur et le statut final.
- 2. Construire idempotency_key = provider + provider_tx_id + meter_id + amount.
- 3. Si idempotency_key est déjà traité avec SUCCESS, retourner le statut existant sans créer une nouvelle recharge.
- 4. Vérifier l'association customer_id/meter_id, statut du compteur, éligibilité à l'auto-recharge et plafonds.
- 5. Demander le token de génération ou l'ordre de recharge au système de prépaiement/HSM selon l'architecture.
- 6. Créer command_id et correlation_id ; pousser la commande au HES/IoT avec une séquence monotone.
- 7. Attendre ACK technique puis ACK métier ; sinon retry exponentiel avec limite N et dead-letter queue.
- 8. En cas d'échec définitif technique, basculer sur le fallback : notification token visible ou ouverture d'un ticket L2.
- 9. Journaliser tous les statuts et notifier client avec statut clair.

ALG-03 Detection compteur offline

- 1. Chaque device publie heartbeat periodique avec timestamp, RSSI, operateur, batterie/alim, firmware.
- 2. Job de controle compare now - last_seen au seuil par modele/zone.
- 3. Si depassement seuil, verifier si incident telecom massif sur la zone.
- 4. Classifier : OFFLINE_INDIVIDUAL, OFFLINE_CLUSTER, TELECOM_SUSPECT, POWER_OUTAGE_SUSPECT.
- 5. Creer ticket seulement si depassement duree ou recurrence ; sinon alerte observation.
- 6. Prioriser selon criticite client, nombre de clients impactes, historique fraude, SLA contractuel.
- 7. Fermer automatiquement si retour heartbeat stable pendant fenetre configuree.

ALG-04 Scoring fraude et anomalies

- 1. Collecter event tamper, ouverture capot, magnetic, reverse energy, baisse brutale kWh, incoherence tension/courant.
- 2. Calculer score par ponderation : tamper direct fort, baisse brutale moyen, offline recurrent moyen, tension incoherente moyen.
- 3. Comparer a baseline historique client et baseline quartier pour limiter faux positifs.
- 4. Classer : LOW observation, MEDIUM revue support, HIGH investigation terrain, CRITICAL SOC + terrain.
- 5. Associer preuve : telemetry, event log, photo intervention, historique commandes.
- 6. Ne jamais couper automatiquement sur score seul sans regle metier validee et procedure conforme.

ALG-05 Priorisation incidents support

- 1. Entrer incident_type, severity, client_segment, zone, age_ticket, recurrence, SLA_due_at.
- 2. Calculer urgency = temps restant SLA + criticite service + impact client + risque securite.
- 3. Calculer score = 40% criticite + 25% SLA + 20% impact zone + 15% recurrence.
- 4. Trier files L1/L2/terrain par score, puis par anciennete.
- 5. Escalader L2 si L1 ne peut confirmer status payment/command/ACK en delai configure.
- 6. Escalader incident majeur si cluster géographique ou volume depasse seuil.

6. Dashboards

6.1 Dashboard client simple

Tuile	Contenu	Valeur
Solde energie	FCFA/kWh restant, statut couleur	Client comprend immediatement son risque de coupure.
Autonomie estimee	jours/heures restants + confiance	Permet d'anticiper recharge.
Recharge rapide	montants favoris + Mobile Money	Reduction friction paiement.
Statut recharge	paiement recu, commande envoyee, compteur credite, token a saisir	Transparence et baisse appels support.
Historique consommation	jour/semaine/mois, comparaison periode precedente	Education energetique.
Alertes	credit faible, incident, coupure quartier, fraude suspectee non affichee au client	Information utile et actionnable.

6.2 Dashboard CIE fournisseur / exploitation

Tuile	Indicateurs	Usage
Parc connecte	installes, online, offline, dernier heartbeat	Vue sante globale.
Recharges	volume, montant, succes, pending, failed, P95 delai	Pilotage service.
Incidents	par type, zone, SLA, backlog L1/L2/terrain	Support et priorisation.
Telecom	operateur, RSSI, APN, latence, pertes	Liaison avec operateurs.
Fraude/anomalies	tamper, offline recurrent, baisse anormale, score	Protection revenu.
Cyber	certificats expirants, auth refusees, firmware non conforme	SOC/RSSI.
Data quality	doublons, donnees manquantes, horodatage incoherent	Fiabilite MDMS.

7. Démarche de deployment sans changement fort

Phase	Objectif	Contenu
Phase 0	Cadrage et gouvernance	Nommer PO/Directeur produit, RSSI, architecte, responsables metier, support, fournisseur. Valider principes non-regression SI existant.
Phase 1	Qualification parc	Inventorier modeles compteurs, ports, protocoles, STS, contraintes securite, telecom, energie, lieux.
Phase 2	Prototype lab	Tester 20-50 compteurs en laboratoire : lecture, injection, fallback, securite, OTA, logs.
Phase 3	Pilote terrain	Installer 100-500 devices sur zones variees, mesurer connectivite, taux succes, support, acceptation client.
Phase 4	Pre-industrialisation	Etendre 2 000-10 000 ; automatiser provisioning, MDM, ITSM, runbooks, supervision.
Phase 5	Industrialisation	Cadencer deployment national par lots, avec GO/NO GO, KPI, incidents majeurs, formation agences.

8. RACI synthetique

Activite	DSI/Produit	Metier	IoT/HES	RSSI	Fournisseur	Support
Architecture cible	A/R	C	C	C	C	C
Qualification compteur	C	C	A/R	C	R	C
Cyber/PKI/HSM	C	C	C	A/R	C	C
App client	A/R	R	C	C	C	C
Support L1/L2	C	C	C	C	C	A/R
Incident majeur	A	C	R	R	R	R
Roadmap produit	A/R	C	C	C	C	C

9. Exigences fournisseur / RFP

Theme	Exigence
Compatibilite	Liste modeles compteurs supportes, protocole, limites, mode fallback.
Securite	Certificats device, chiffrement, signature firmware, OTA, gestion secrets, audit.
Exploitation	Logs, metrics, traces, supervision, runbooks, SLA, support L3.
Integration	API OpenAPI, webhooks, Kafka/MQTT, export MDMS, ITSM, IAM.
Reversibilite	Export donnees, documentation, transfert competences, garanties fin contrat.
Performance	Capacite device, messages/seconde, latence commande, retry, HA/DRP.
Qualite	FAT/SAT, tests terrain, certification STS/DLMS si applicable, rapports.

10. Sources et limites

CIE PEPT - fonctionnement du compteur - La page publique CIE mentionne les smart meters, la télégestion, le rechargement et le traitement rapide des anomalies à distance. URL: <https://www.cie.ci/pept/fonctionnement-du-compteur>

Ma CIE en ligne - La page publique présente le suivi de la consommation, l'assistance, les demandes en ligne et le signalement d'incident. URL: <https://macieenligne.ci/>

DLMS/COSEM - DLMS/COSEM est le standard global pour le metering intelligent, avec un modèle d'objet, une couche applicative et des profils de communication. URL: https://www.dlms.dev/dlms_cosem/

STS Association - STS décrit un système de message sécurisé pour le transfert de tokens de prépaiement entre le point de vente et le compteur. URL: <https://www.sts.org.za/>

Limite importante : les protocoles réels du parc CIE, les modèles de compteurs, les API Mobile Money, les exigences réglementaires et les capacités du SI existant doivent être confirmés par des ateliers CIE/fournisseurs. Toute injection automatique dans le compteur est conditionnée par la compatibilité technique et sécuritaire du compteur et du système de prépaiement.

Addendum V2 - Spécifications Java, NodeJS, algorithmes et résumé

Direction Générale

Objectif : compléter le dossier d'architecture Smart Prepaid sans changement massif des compteurs existants avec les spécifications de code, les algorithmes opérationnels et un résumé exécutif directement exploitable par la Direction Générale.

1. Résumé exécutif pour la Direction Générale

Axe	Message exécutif
Décision proposée	Lancer un programme CIE Smart Prepaid Retrofit piloté par la DSI, sans remplacement massif des compteurs, via un module connecté non intrusif, une plateforme HES/MDMS et des parcours digitaux client/CIE.
Promesse client	Suivi simple du crédit, consommation visible, alertes avant coupure, paiement Mobile Money, injection automatique du code/token lorsque la compatibilité compteur est confirmée.
Promesse CIE	Vision temps réel du parc, baisse des appels et déplacements, incidents pilotés par la donnée, meilleure traçabilité de la recharge, détection proactive des anomalies et fraudes.
Approche industrielle	Procéder par qualification, prototype, pilote, pré-industrialisation puis généralisation par vagues, avec critères de passage mesurables à chaque étape.
Risque majeur à maîtriser	Hétérogénéité des compteurs et interfaces disponibles. La solution doit donc commencer par une qualification technique terrain, sans hypothèse non vérifiée sur DLMS/COSEM, STS ou ports physiques.
Principe de gouvernance	La CIE peut acheter certaines briques spécialisées, mais doit conserver la maîtrise de l'architecture, des données, des API, des règles métier, de la cybersécurité, du monitoring et de la réversibilité fournisseur.

2. Spécifications détaillées du code Java / Spring Boot

Le périmètre Java couvre le cœur critique transactionnel : orchestration des recharges, commandes compteur, traçabilité, règles métier, persistance, audit, sécurité API interne et intégration HES/MDMS. Les services Java doivent être conçus en architecture hexagonale afin de séparer domaine, ports, adaptateurs et infrastructure.

Service Java	Responsabilité	Stack recommandée
recharge-core-service	Valider la demande de recharge, créer la transaction, piloter paiement, token, commande et statut final.	Java 21, Spring Boot, PostgreSQL, Kafka, API REST
command-service	Créer, suivre et rejouer les commandes vers le HES ou le dongle.	Java 21, Spring Boot, Kafka, MQTT/HES adapter
meter-registry-service	Gérer la référence client, compteur, device, SIM/eSIM, modèle compteur et compatibilité.	Java 21, JPA, PostgreSQL
incident-service	Transformer les événements en incidents qualifiés et tickets support.	Java 21, Kafka consumer, ITSM adapter
rules-engine-service	Évaluer seuils crédit, prédiction coupure, fraude simple, retry, escalade.	Java 21, Drools optionnel ou règles code/config
audit-service	Journaliser les actions critiques et produire la traçabilité complète.	Java 21, append-only log, PostgreSQL/OpenSearch

2.1 Structure de packages Java recommandée

```
ci.cie.smartprepaid
  recharge
    domain
      model: Recharge, Payment, Token, MeterCommand, CommandStatus
      service: RechargeDomainService, AutoRechargePolicy
      event: RechargeRequested, PaymentConfirmed, TokenGenerated, CommandAcknowledged
    application
      usecase: StartRechargeUseCase, ConfirmPaymentUseCase, PushTokenUseCase, ReconcileRechargeUseCase
      dto: RechargeRequest, RechargeResponse, CommandStatusResponse
    ports
      in: RechargeApiPort, ReconciliationJobPort
      out: PaymentProviderPort, TokenProviderPort, HesCommandPort, MeterRepositoryPort, EventPublisherPort
  infrastructure
    api: RechargeController
    persistence: JpaRechargeRepository, JpaMeterRepository
    messaging: KafkaEventPublisher, KafkaEventConsumer
    security: OAuth2JwtSecurityConfig
    observability: CorrelationIdFilter, MetricsConfig
```

2.2 Contrats API Java principaux

Endpoint	But	Entrée	Sortie
POST /api/v1/recharges	Créer une recharge manuelle ou auto-recharge.	customerId, meterId, amount, channel, paymentProvider, idempotencyKey	rechargeId, status, correlationId
GET /api/v1/recharges/{id}	Consulter l'état de bout en bout.	rechargeId	paymentStatus, tokenStatus, commandStatus, finalStatus
POST /api/v1/commands/{id}/retry	Relancer une commande échouée selon politique autorisée.	commandId, reason, operatorId	retryStatus
GET /api/v1/meters/{meterId}/status	Consulter le statut compteur/device.	meterId	onlineStatus, creditBalance, lastReadingAt, alarmStatus
POST /api/v1/incidents	Créer incident depuis événement technique ou métier.	eventType, meterId, severity, payload	incidentId, priority, assignmentGroup

2.3 Exemple de squelette Java Spring Boot

```

@RestController
@RequestMapping("/api/v1/recharges")
@RequiredArgsConstructor
class RechargeController {
    private final StartRechargeUseCase startRecharge;

    @PostMapping
    ResponseEntity<RechargeResponse> create(@Valid @RequestBody RechargeRequest request,
                                             @RequestHeader("Idempotency-Key") String idempotencyKey,
                                             @RequestHeader("X-Correlation-Id") String correlationId) {
        RechargeCommand command = RechargeCommand.from(request, idempotencyKey, correlationId);
        RechargeResult result = startRecharge.execute(command);
        return ResponseEntity.accepted().body(RechargeResponse.from(result));
    }
}

@Service
@RequiredArgsConstructor
class StartRechargeUseCase {
    private final MeterRepository meterRepository;
    private final PaymentProviderPort paymentProvider;
    private final TokenProviderPort tokenProvider;
    private final HesCommandPort hesCommandPort;
    private final EventPublisherPort eventPublisher;

    @Transactional
    public RechargeResult execute(RechargeCommand command) {
        Meter meter = meterRepository.findActiveMeter(command.meterId());
        meter.assertRechargeCompatible();
        Recharge recharge = Recharge.start(command, meter);
        PaymentResult payment = paymentProvider.requestDebit(recharge.paymentIntent());
        recharge.attachPayment(payment);
        eventPublisher.publish(RechargeRequested.from(recharge));
        return RechargeResult.accepted(recharge);
    }
}

```

3. Spécifications détaillées du code NodeJS / TypeScript

Le périmètre NodeJS/TypeScript couvre les services digitaux, les BFF, les notifications, les connecteurs Mobile Money non critiques et l'expérience client. Les flux critiques restent orchestrés par Java, tandis que NodeJS accélère l'intégration, l'exposition API et les canaux digitaux.

Service NodeJS	Responsabilité	Stack recommandée
customer-bff	Agrégation des données client pour mobile/web : solde, conso, alertes, historique, auto-recharge.	NestJS, TypeScript, REST/GraphQL optionnel
notification-service	Routage SMS, WhatsApp, push app, e-mail selon préférence client et criticité.	NestJS, BullMQ/Redis, providers SMS/WhatsApp
payment-adapter-service	Connecteurs Mobile Money et normalisation des callbacks.	NestJS, webhooks, idempotency store
portal-cie-bff	Vue opérateur CIE : incidents, compteurs, recharges, commandes, statuts.	NestJS, React BFF, API Gateway
dashboard-api	APIs agrégées pour tableaux de bord client et exploitation CIE.	NodeJS, cache Redis, lecture API/DB read model

3.1 Structure TypeScript recommandée

```
src/  
  main.ts  
  app.module.ts  
  common/  
    filters/http-exception.filter.ts  
    interceptors/correlation-id.interceptor.ts  
    guards/jwt-auth.guard.ts  
    dto/pagination.dto.ts  
  customer-bff/  
    customer-bff.module.ts  
    controllers/customer-dashboard.controller.ts  
    services/customer-dashboard.service.ts  
    clients/recharge-core.client.ts  
    clients/meter-status.client.ts  
  notifications/  
    notification.module.ts  
    controllers/notification-webhook.controller.ts  
    services/notification-router.service.ts  
    providers/sms.provider.ts  
    providers/whatsapp.provider.ts  
    providers/push.provider.ts  
  payments/  
    payment-adapter.module.ts  
    controllers/mobile-money-webhook.controller.ts  
    services/payment-normalizer.service.ts  
    providers/orange-money.provider.ts  
    providers/mtn-money.provider.ts  
    providers/wave.provider.ts
```

3.2 Exemple de squelette NodeJS / NestJS

```
@Controller('/api/v1/customer-dashboard')  
@UseGuards(JwtAuthGuard)  
export class CustomerDashboardController {  
  constructor(private readonly dashboard: CustomerDashboardService) {}  
  
  @Get('/:customerId')  
  async getDashboard(@Param('customerId') customerId: string,  
    @Req() req: Request): Promise<CustomerDashboardDto> {  
    return this.dashboard.getDashboard(customerId, req.headers['x-correlation-id'] as string);  
  }  
}  
  
@Injectable()  
export class CustomerDashboardService {  
  constructor(private readonly rechargeCore: RechargeCoreClient,  
    private readonly meterStatus: MeterStatusClient,  
    private readonly cache: CacheService) {}  
  
  async getDashboard(customerId: string, correlationId: string): Promise<CustomerDashboardDto> {  
    const cacheKey = `dashboard:${customerId}`;  
    const cached = await this.cache.get<CustomerDashboardDto>(cacheKey);  
    if (cached) return cached;  
  
    const [meters, alerts, recharges] = await Promise.all([  
      this.meterStatus.getMeters(customerId, correlationId),  
      this.rechargeCore.getActiveAlerts(customerId, correlationId),  
      this.rechargeCore.getRecentRecharges(customerId, correlationId)  
    ]);  
  
    const result = CustomerDashboardDto.from(meters, alerts, recharges);  
    await this.cache.set(cacheKey, result, 60);  
    return result;  
  }  
}
```

4. Algorithme complet de prédiction d'épuisement du crédit

L'algorithme doit rester explicable, robuste et paramétrable. Pour la phase pilote, il est préférable d'utiliser un modèle déterministe basé sur moyenne mobile, pondération récente, jours atypiques et marge de sécurité. Les modèles IA peuvent venir ensuite lorsque l'historique est suffisant.

Entrée	Description
credit_balance	Crédit restant en FCFA ou kWh selon référentiel.
daily_consumption_30d	Consommation journalière historique sur 30 jours.
daily_consumption_7d	Consommation journalière récente sur 7 jours.
meter_status	Online/offline, dernière lecture, qualité donnée.
customer_segment	Résidentiel, commerce, administration, industriel léger.
seasonality_factor	Facteur horaire/jour/semaine si disponible.
safety_margin	Marge de prudence pour éviter une alerte trop tardive.

```
ALGORITHM PredictCreditExhaustion
INPUTS:
  credit_balance
  consumption_history[30 days]
  recent_consumption[7 days]
  meter_status
  minimum_data_quality_threshold
  alert_thresholds = {J7, J3, J1, H3}
  safety_margin_percent

STEPS:
  1. Validate input quality
    IF meter_status.last_reading_age > configured_max_age THEN
      return Prediction(status = DEGRADED, reason = STALE_METER_DATA)
    IF count(valid consumption_history) < minimum_data_quality_threshold THEN
      use fallback profile by customer_segment

  2. Clean data
    Remove negative values
    Cap extreme outliers using percentile P95 or business cap
    Mark abnormal days separately rather than deleting all suspicious data

  3. Calculate baseline consumption
    avg_30d = average(valid consumption_history)
    avg_7d = average(valid recent_consumption)
    weighted_daily_consumption = (0.65 * avg_7d) + (0.35 * avg_30d)

  4. Apply seasonality and margin
    adjusted_daily_consumption = weighted_daily_consumption * seasonality_factor
    prudent_daily_consumption = adjusted_daily_consumption * (1 + safety_margin_percent)

  5. Estimate autonomy
    IF prudent_daily_consumption <= 0 THEN
      return Prediction(status = UNKNOWN, reason = ZERO_CONSUMPTION)
    autonomy_days = credit_balance / prudent_daily_consumption
    estimated_exhaustion_date = now + autonomy_days

  6. Determine alert level
    IF autonomy_days <= 0.125 THEN level = IMMEDIATE_H3
    ELSE IF autonomy_days <= 1 THEN level = CRITICAL_J1
    ELSE IF autonomy_days <= 3 THEN level = WARNING_J3
    ELSE IF autonomy_days <= 7 THEN level = INFO_J7
    ELSE level = NORMAL

  7. Publish result
    Save prediction with version, input hash and correlation_id
    Publish LOW_CREDIT_PREDICTED event if level != NORMAL
    Notify customer according to preference and severity

OUTPUT:
  prediction_id, autonomy_days, estimated_exhaustion_date, alert_level, confidence, data_quality_status
```

5. Algorithme complet d'auto-recharge

L'auto-recharge doit être sécurisée, explicite pour le client, désactivable, plafonnée, traçable et résistante aux doublons. Chaque étape doit être idempotente pour éviter un double débit Mobile Money ou une double injection de crédit.

Bloc	Exigence
Consentement client	Auto-recharge activée explicitement, méthode de paiement autorisée, plafond journalier/mensuel.
Déclencheur	Crédit restant sous seuil ou prédiction de coupure sous horizon configuré.
Sécurité	Idempotency key, correlation_id, contrôle anti-doublon, audit complet.
Paieement	Demande débit Mobile Money puis callback confirmé ou rejeté.
Token/commande	Génération token STS ou commande compatible, push vers HES/dongle/compteur.
Fallback	Si injection automatique impossible, envoi du code au client avec procédure simple.

ALGORITHM AutoRecharge

INPUTS:

```
customer_id
meter_id
current_credit_balance
prediction_result
auto_recharge_profile = {enabled, threshold, amount, payment_method, daily_cap, monthly_cap}
idempotency_window
```

STEPS:

1. Eligibility control

```
IF auto_recharge_profile.enabled == false THEN STOP
IF meter is inactive or blocked THEN STOP with reason METER_NOT_ELIGIBLE
IF payment_method not valid THEN notify customer PAYMENT_METHOD_INVALID and STOP
```
2. Trigger evaluation

```
trigger_by_threshold = current_credit_balance <= auto_recharge_profile.threshold
trigger_by_prediction = prediction_result.alert_level in {CRITICAL_J1, IMMEDIATE_H3}
IF not trigger_by_threshold AND not trigger_by_prediction THEN STOP
```
3. Anti-duplicate control

```
idempotency_key = hash(customer_id, meter_id, current_day, auto_recharge_profile.amount, trigger_type)
IF recharge exists with same idempotency_key in idempotency_window THEN STOP with existing recharge_id
```
4. Limits and risk control

```
IF daily_total(customer_id) + amount > daily_cap THEN notify customer CAP_REACHED and STOP
IF monthly_total(customer_id) + amount > monthly_cap THEN notify customer CAP_REACHED and STOP
IF fraud_flag == HIGH THEN create incident and STOP_PENDING_REVIEW
```
5. Create recharge transaction

```
status = CREATED
persist recharge with correlation_id, idempotency_key, meter_id, amount
publish AUTO_RECHARGE_REQUESTED
```
6. Request Mobile Money debit

```
payment_request = paymentProvider.debit(payment_method, amount, idempotency_key)
status = PAYMENT_PENDING
wait for callback or poll according to provider contract
```
7. Handle payment callback

```
IF callback == FAILED THEN
    status = PAYMENT_FAILED
    notify customer PAYMENT_FAILED
    STOP
IF callback == CONFIRMED THEN
    status = PAYMENT_CONFIRMED
    publish PAYMENT_CONFIRMED
```
8. Generate token or command

```
token = tokenProvider.generate(meter_id, amount, tariff, sequence, security_context)
status = TOKEN_GENERATED
persist token fingerprint only if sensitive token handling requires masking
```
9. Push to meter

```
command = hesCommandPort.pushCredit(meter_id, token, correlation_id)
status = COMMAND_SENT
wait for ACK within configured timeout
```

```

10. Confirm final status
  IF ACK == SUCCESS THEN
    status = CREDIT_APPLIED
    notify customer RECHARGE_SUCCESS
    update dashboard and event stream
  ELSE IF ACK == TIMEOUT THEN
    status = COMMAND_UNKNOWN
    retry according to retry policy
    if retries exhausted: create L2 incident and send fallback token if allowed
  ELSE IF ACK == REJECTED THEN
    status = COMMAND_REJECTED
    create incident with error_code
    notify customer SUPPORT_REQUIRED

```

```

OUTPUT:
  recharge_id, final_status, payment_status, command_status, customer_message, incident_id_if_any

```

6. Tableau de bord client et CIE - exigences fonctionnelles simples

Vue	Contenu minimum
Client - Accueil	Crédit disponible, autonomie estimée, statut compteur, bouton Recharger, état auto-recharge, dernière mise à jour.
Client - Consommation	Consommation jour/semaine/mois, courbe simple, montant estimé, comparaison période précédente.
Client - Alertes	Crédit faible, paiement échoué, coupure imminente, recharge réussie, incident compteur.
CIE - Supervision	Compteurs connectés/offline, recharges en cours, taux de succès, files de commandes, alertes critiques.
CIE - Incidents	Priorité, type incident, zone, compteur, client, statut, SLA, groupe affecté.
CIE - Support	Recherche par client, compteur, paiement, recharge, command_id et correlation id.

7. Critères de passage en pilote et industrialisation

Jalon	Critères
Prototype	Lecture crédit fiable, réception événement, recharge simulée, dashboard minimal, journalisation bout en bout.
Pilote	Paiement réel contrôlé, notification client, incident automatique, supervision CIE, mesure SLA, tests de charge.
Pré-industrialisation	Multi-zones, multi-compteurs, procédures RUN, support L1/L2/L3, sécurité auditée, réversibilité fournisseur.
Déploiement national	Déploiement par vagues, capacité réseau validée, observabilité, gouvernance data, budget RUN récurrent.